Classification of US Supreme Court Opinions

Team: Gowtham S and Harsh Venkat Sai Kalekar | UE24CS352A Machine Learning Mini-Project

Code: https://github.com/gowth139am-lab/legal-text-classification

1. Problem Statement

Supreme Court opinions must be classified and cataloged so that lawyers can find earlier cases on the same legal issue. Doing this by hand is slow and tedious. Our project, based on the paper *Classification of Legal Text* by Krithika Iyer, asks whether a machine learning model can read the text of an opinion and predict its issue area automatically (for example Criminal Procedure, Civil Rights or First Amendment).

2. Dataset

We used the `SupremeCourt` dataset from the Textacy library, which is derived from the Supreme Court Database (scdb.wustl.edu). It holds 8,419 opinions from 1946 onwards, each labeled by experts with an issue area. Opinions are long: about 6,960 words on average, with a maximum of over 87,000 words.

Cleaning removed 26 rows: 23 cases with no real label, 2 with empty text, and 1 case from a class with fewer than 5 examples. This left 8,393 opinions in **13 classes**. We made a stratified 80/20 split (random_state=42): 6,714 training and 1,679 test cases. The classes are imbalanced. Criminal Procedure is the largest (about 23% of cases), while Miscellaneous has only 4 test cases.

3. Approach

Each model turns an opinion into numbers and then uses Logistic Regression (with `class_weight="balanced"` to handle imbalance) to predict the issue area. We compared three representations:

- **TF-IDF + Logistic Regression:** word and word-pair frequencies weighted by how distinctive they are. This is a baseline we added.
- **LDA + Logistic Regression:** a 50-topic Latent Dirichlet Allocation model (Gensim) gives each opinion a vector of topic probabilities. This is the paper's baseline.
- **Doc2Vec + Logistic Regression:** a 100-dimensional Gensim document vector per opinion. This was the paper's best model.

We did not use BERT. Opinions average about 7,000 words, far beyond its 512-token limit, and the paper's own BERT experiment ran out of memory.

4. Implementation

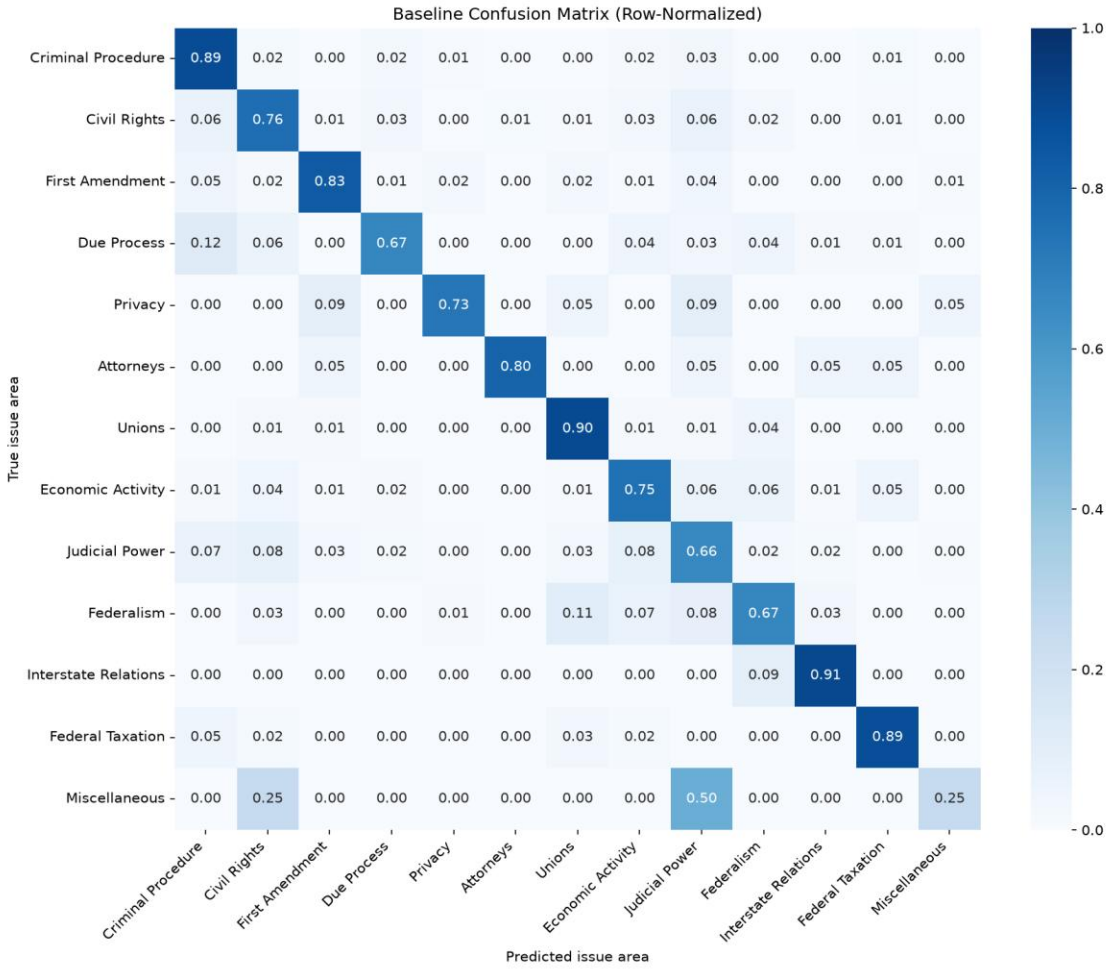
The project is written in Python 3.11 in VS Code and uses Gensim, scikit-learn, pandas and Streamlit. The pipeline is split into scripts in `src/`: `load_data.py` downloads and saves the data, `preprocess.py` cleans the text and creates the split, `train_baseline.py`, `train_lda.py` and `train_doc2vec.py` train and evaluate the three models, and `evaluate.py` builds the comparison table, chart and confusion

matrix. Vocabularies and models are fitted on the training set only. A Streamlit app (`app.py`) lets a user paste an opinion or load a random test case and shows the predicted issue area with the top five class probabilities.

5. Results

All results are on the 1,679-case test set.

Model	Accuracy	Macro-F1
Majority-class guess	0.229	0.029
LDA + Logistic Regression	0.631	0.522
Doc2Vec + Logistic Regression	0.630	0.539
TF-IDF + Logistic Regression	**0.783**	**0.706**



1 Confusion matrix of the TF-IDF + Logistic Regression model on the test set (rows normalized).

TF-IDF performed best by about 15 points. LDA and Doc2Vec are essentially tied: their accuracy differs by about 0.001, which is only 2 test cases. For TF-IDF, the strongest classes were Criminal Procedure (F1 0.88) and First Amendment (0.86). The weakest were Miscellaneous (0.22, only 4 test cases), Federalism (0.61), Interstate Relations (0.65) and Judicial Power (0.68), which overlaps with Economic Activity and Due Process.

6. Conclusions

- Classifying legal opinions into issue areas is feasible. All three models are far above the majority-class baseline of 0.229.
- Simple TF-IDF features beat the topic-model and document-embedding approaches. A likely reason is that LDA and Doc2Vec compress each long opinion into 50 or 100 numbers and lose detail that individual words keep.
- We could not reproduce the paper's finding that Doc2Vec clearly beats LDA. Our results are not directly comparable with the paper's, because our dataset version, class set and split differ.
- **Limitations:** a single train/test split with no cross-validation, very small classes whose scores are noisy, and single-label prediction even though many cases cover several topics.
- **Future work:** cross-validation, models for long documents such as Longformer, and a similarity-search feature to find related cases.